Regular Expressions and Automata

Theorem:

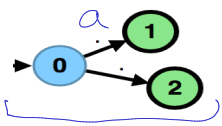
Let L be a language. There exists a regular expression that describes it if and only if there exists a finite automaton that accepts it.

Algorithms:

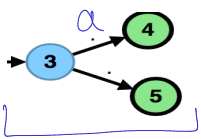
- regular expression → automaton (important!)
- automaton → regular expression (cool)

Recursive Constructions

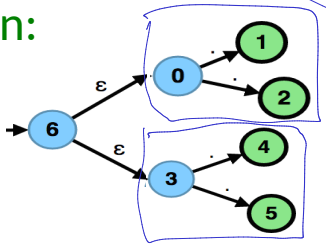
A_1:



A_2:



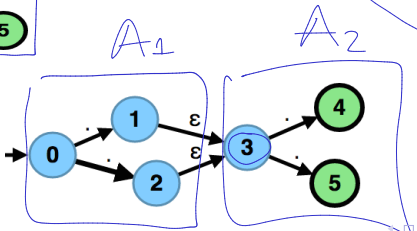
Union:



$r_1 \cup r_2$

$+$

Concatenation:

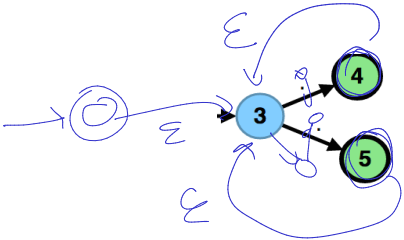


$r_1 \quad r_2$

$w_1 \quad w_2$

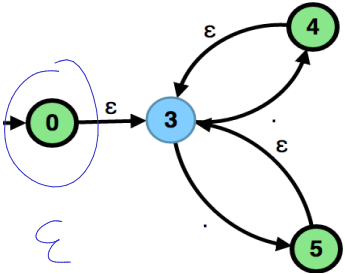
w

Recursive Constructions



r

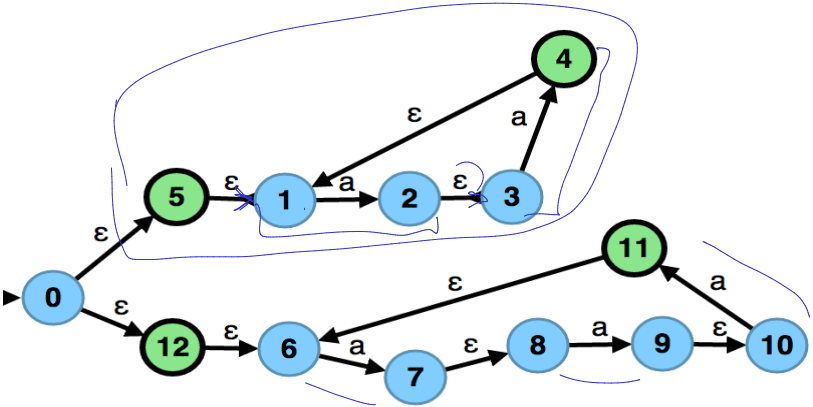
Star:



$\varepsilon \in \boxed{r}^*$

Exercise: (aa)* | (aaa)*

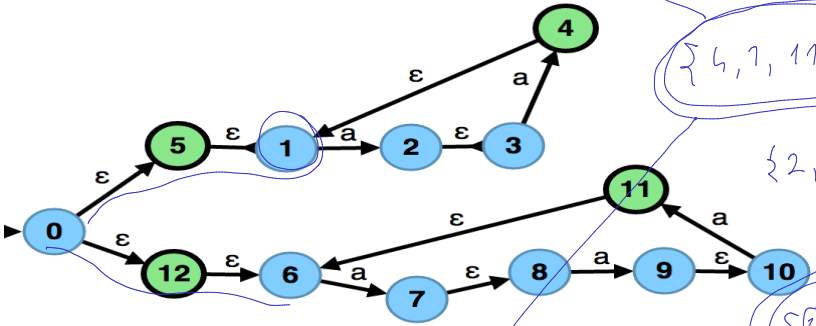
- Construct an NFA for the regular expression



NFAs to DFAs (Determinization)

- keep track of a set of all possible states in which the automaton could be
- view this finite set as one state of new automaton

NFA to DFA Conversion



ε closure

{4, 1, 11, 6}

a

{2, 3, 9, 10}

a

{4, 1, 7, 8}

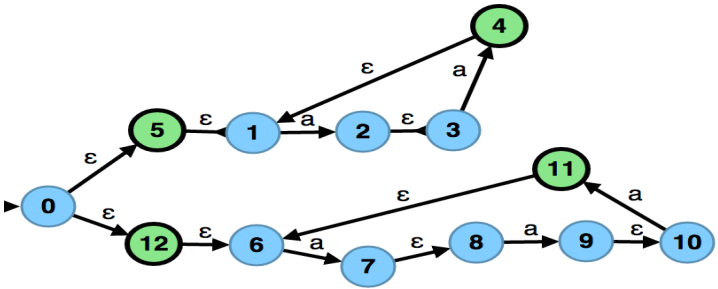
a

Possible states of the DFA: 2^Q

{ { } , { 0},...{12}, {0,1}, ...,{0,12}, ...{12, 12}, {0,1,2} ..., { 0,1,2...,12 } }

{0, 5, 1, 12, 6} $\xrightarrow{a}$ {2, 3, 7, 8} $\xrightarrow{a}$ {4, 1, 9, 10} $\xrightarrow{a}$ {2, 3, 11, 6}

NFA to DFA Conversion



Epsilon Closure

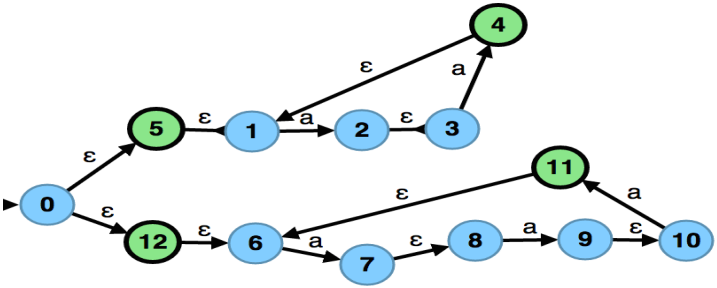
- All states reachable from a state through epsilon
- $q \in E(q)$
- If $q_1 \in E(q)$ and $\delta(q_1, \epsilon, q_2)$ then $q_2 \in E(q)$

E(0) = { } E(1) = { } E(2) = { }

NFA to DFA Conversion

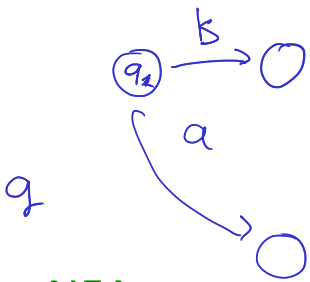
- DFA: $(\Sigma, 2^Q, q_0', \delta', F')$
- $q_0' = E(q_0)$
- $\delta'(q', a) = \bigcup_{\{\exists q_1 \in q',\ \delta(q_1, a, q_2)\}} E(q_2)$
- $F' = \{\, q' \mid q' \in 2^Q, q' \cap F \neq \emptyset \}$

NFA to DFA Conversion through Examle



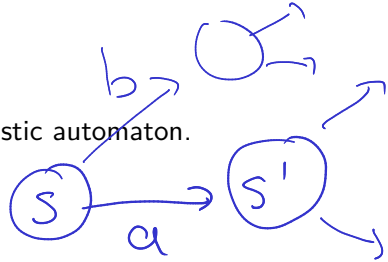
Clarifications

- what happens if a transition on an alphabet 'a' is not defined for a state 'q' ?
- $\delta'(\{q\}, a) = \emptyset$
- $\delta'(\emptyset, a) = \emptyset$



- Empty set represents a state in the NFA
- It is a trap/sink state: a state that has self-loops for all symbols, and is non-accepting.

Relationship between Automata and Derivatives



Regular expressions can be viewed as states of a deterministic automaton.

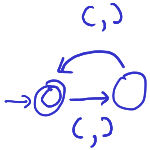
For $a \in A$,
$s' = \delta(s, a)$ corresponds to $s' = \partial_a s$.

Problem: there can be an infinite number of syntactically different regular expressions, so the automaton is infinite (states may get bigger and bigger).

Solution: using properties of regular expression (e.g. $+$ is associative and commutative), we can identify those expressions that denote the same language. This gives back a finite state machine.

This is the essence of ZipLex tool that we use in the labs.

Grammars



Expressive Power of Automata

For which of the following languages can you find an automaton or regular expression:

- Sequence of open or closed parentheses of even length? E.g. (), ((,)),)())))(, ...
- as many digits before as after decimal point?
- Sequence of balanced parentheses

  ((()) ()) - balanced

  ( )) (( ) - not balanced
- Comments from // until LF
- Nested comments like /* ... /* */ ... */

Expressive Power of Automata

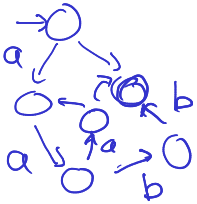
For which of the following languages can you find an automaton or regular expression:

- Sequence of open or closed parentheses of even length? E.g. (), ((,)),)()))(, ... yes
- as many digits before as after decimal point? No
- Sequence of balanced parentheses
  ((()) ()) - balanced
  ( )) (( ) - not balanced
  No
- Comments from // until LF Yes
- Nested comments like /* ... /* */ ... */ No

Automaton that Claims to Recognize

K states

$\in \{ a^n b^n \mid n >= 0 \}$



Make the automaton deterministic

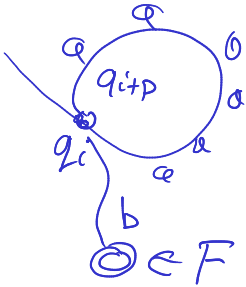
Let the resulting DFA have K states, |Q|=K

Feed it a, aa, aaa, ... Let q_i be state after reading a^i

$q_0, q_1, q_2, \ldots, q_K$

This sequence has length K+1 -> a state must repeat

$q_i = q_{i+p}$ $\quad$ $p > 0$



Then the automaton should accept $a^{i+p} b^{i+p}$.

But then it must also accept

$$\boxed{a^i b^{i+p}}$$

because it is in state after reading a^i as after a^{i+p}.

So it does not accept the given language.

Limitations of Regular Languages

- Every automaton can be made deterministic
- Automaton has finite memory, cannot count
- Deterministic automaton from a given state behaves always the same
- If a string is too long, deterministic automaton will repeat its behavior

Pumping Lemma

If L is a regular language, then there exists a positive integer p (the pumping length) such that every string $s \in L$ for which $|s| \geq p$, can be partitioned into three pieces, $s = x\,y\,z$, such that

- $|y| > 0$
- $|xy| \leq p$
- $\forall i \geq 0.\ xy^iz \in L$

$i=0$ $i=100$

a

Let's try again: { $a^n b^n$ | n >= 0 }

Finite State Automata are Limited

Let us use (context-free) **grammars**!

Context Free Grammar for $a^n b^n$

$S ::= \varepsilon$ - first rule of this grammar

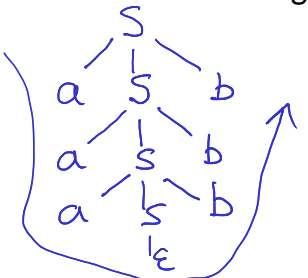
$S ::= a\ S\ b$ - second rule of this grammar.

Example of a derivation

$S \Rightarrow aSb \Rightarrow a\ aSb\ b \Rightarrow aa\ aSb\ bb \Rightarrow aaabbb$

Parse tree:

leaves give us the result



a a a b b b

Context-Free Grammars

G = (A, N, S, R)

- A - terminals (alphabet for generated words w ∈ A*)
- N - non-terminals – symbols with (recursive) definitions
- Grammar rules in R are pairs (n,v), written
  n::=v where
  n ∈ N is a non-terminal
  v ∈ (A U N)* - sequence of terminals and non-terminals

A derivation in G starts from the starting symbol S

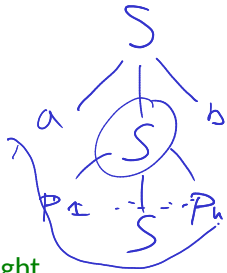
- Each step replaces a non-terminal with one of its right hand sides

Example from before: G = ({a,b}, {S}, S, {(S,ε), (S,aSb)})

A

S ::= ε
S ::= aSb

Parse Tree



Given a grammar G = (A, N, S, R), t is a **parse tree** of G iff t is a node-labelled tree with ordered children that satisfies:

- root is labeled by S
- leaves are labelled by elements of A
- each non-leaf node is labelled by an element of N
- for each non-leaf node labelled by n whose children left to right are labelled by $p_1 \ldots p_n$, we have a rule $(n ::= p_1 \ldots p_n) \in R$

Yield of a parse tree t is the unique word in A^* obtained by reading the leaves of t from left to right

Language of a grammar G = words of all yields of parse trees of G

L(G) = {yield(t) | isParseTree(G,t)}

$w \in L(G) \quad \Leftrightarrow \quad \exists t.\ w = yield(t) \wedge isParseTree(G,t)$

isParseTree - **easy** to check condition, given t

Harder: know *if for a word there* ***exists*** *a parse tree*